

Block Ciphers, AES and Integrity

CSC 3114: Cryptology and Coding Theory | Take-Home Assignment (Test 2) | Group 4

#	Name	Registration No.	Student No.
1	NANSEREKO HOUSNAH	24/U/09631/EVE	2400709631
2	SUUNA RAYMOND	24/U/11403/EVE	2400711403
3	NABUKEERA SUMAYAH KASWA	24/U/07899/EVE	2400707899

Video presentation:

<https://drive.google.com/file/d/1TARreFrYcpibNIpK1Uo6iej6iJCPToCY/view?usp=sharing>

Tools: Python 3 with PyCryptodome, PyCA cryptography and the standard library hashlib/hmac.

Task 1: Theory and Cipher Design

a) Confusion, diffusion and the avalanche effect

Confusion hides the relationship between the key and the ciphertext so the key cannot be worked out from the output. In AES it comes from **SubBytes**, a non-linear S-box substitution. **Diffusion** spreads each plaintext bit across many ciphertext bits so patterns disappear. AES achieves it with **ShiftRows** and **MixColumns**. Together, over repeated rounds, they produce the **avalanche effect**: changing one input bit should change about half of the output bits.

b) DES vs AES

Property	DES	AES
Block size	64 bits	128 bits
Key size	56 effective bits	128, 192 or 256 bits
Rounds	16	10, 12 or 14
Structure	Feistel network	Substitution-permutation network
Security today	Insecure: key small enough to brute-force	Secure with a proper key size and mode

c) ECB vs CBC vs CTR

ECB encrypts each block independently, so identical blocks give identical ciphertext and patterns leak. **CBC** XORs each block with the previous ciphertext block and needs padding. **CTR** XORs the plaintext with an encrypted counter, so it needs no padding. An **IV/nonce** gives each encryption a fresh start so the same plaintext never encrypts the same way twice. **Misuse to avoid:** never reuse a CTR nonce with the same key. It repeats the keystream and exposes the XOR of the plaintexts.

Task 2: Python Avalanche Experiment

AES-128 with key 1234567890abcdef encrypted the block Sixteen byte txt. We flipped one plaintext bit at a time (bits 0 to 9) and counted how many of the 128 ciphertext bits changed:

```

PS C:\Users\SUMAYAH\Desktop\cryptography26> & c:\Users\SUMAYAH\anaconda3\envs\ml_env\python.exe c:/Users/SUMAYAH/Desktop/
cryptography26/block_ciphers_aes_and_des/task2_avalanche/avalanche.py
-----
Bit Flipped   Changed Bits   Percentage
0             59            46.09%
1             63            49.22%
2             59            46.09%
3             68            53.12%
4             73            57.03%
5             64            50.00%
6             64            50.00%
7             59            46.09%
8             66            51.56%
9             64            50.00%

Average changed ciphertext bits: 63.9
Average percentage of changed ciphertext bits: 49.92 %
PS C:\Users\SUMAYAH\Desktop\cryptography26>

```

Interpretation: one flipped bit changed **49.92%** of the ciphertext on average, almost exactly the ideal 50%, and every trial fell between 46% and 57%. So the ciphertexts of near-identical plaintexts look unrelated. An attacker learns nothing about how similar two inputs are and cannot recover the key by making small changes and watching the output.

Task 3: AES Modes in Practice

Message: The quick brown fox jumps over the lazy dog!! (45 bytes), with a fresh random AES-256 key. CBC used a fresh random IV with PKCS7 padding. CTR used a fresh random nonce/counter.

```

=====
Task 3: AES Modes in Practice
=====
Message       : The quick brown fox jumps over the lazy dog!!
Message length : 45 bytes
Key (hex)     : d0b2cae1e52b791f45649ab2c366a0debe36d7cf5c1e9a3f1d5837b52a73036a

--- AES-CBC ---
IV (hex)      : 02e8b4466e8f1dd9707af29efe929c10
Ciphertext (hex) : bcdd9274928f58a1b1637ca8239920aba4391c0453a6295fd1d5ca36f62eacf88cf413cd24926f98581a28873beb854e
Ciphertext length : 48 bytes
Recovered     : The quick brown fox jumps over the lazy dog!!
Exact match   : True

--- AES-CTR ---
Nonce (hex)   : fa8e846ccdc91224748cebc37892b21b
Ciphertext (hex) : 0bcc629d974ab7d4710f685cd99f40c490f7930c246500fbe563029b4319e61cda4478a28b10bfd98018aac26e
Ciphertext length : 45 bytes
Recovered     : The quick brown fox jumps over the lazy dog!!
Exact match   : True

--- Summary ---
CBC ciphertext length: 48 bytes (padded from 45 to a multiple of 16)
CTR ciphertext length: 45 bytes (same as plaintext, no padding)
Both plaintexts recovered exactly: PASS

```

Observations

- **Padding and length:** CBC works on whole 16-byte blocks, so the 45-byte message grew to 48 bytes. CTR acts as a stream cipher, so the ciphertext is exactly 45 bytes. Neither mode hides the message length.
- **Parallelism:** CBC encryption is sequential because each block depends on the previous one. CTR blocks are independent, so CTR encrypts and decrypts in parallel and allows random access.
- **No integrity:** flipping one ciphertext bit in CTR flips exactly that plaintext bit, and decryption still succeeds. Neither mode detects tampering, which motivates Task 4.

Task 4: Integrity and Tampering

We hashed the message and computed an HMAC-SHA256 tag with a random 32-byte key, then changed one character and recomputed both:

```
PS C:\Users\raymo\Desktop\cryptophy26> & c:\Users\raymo\AppData\Local\Python\pythoncore-3.14-64\python
.exe c:/Users/raymo/Desktop/cryptophy26/block_ciphers_aes_and_des/task4_integrity/integrity.py
=====
Task 4: Integrity and Tampering
=====
Original message : The quick brown fox jumps over the lazy dog!!
Modified message : The quick brown box jumps over the lazy dog!!
Changed position : index 16 ('f' -> 'b')
HMAC key (hex)   : b42a77019803c965a16a20d23081e4b9a4e05399c4795d4f9bf80d2afd8042f0

--- SHA-256 ---
Original          : e8af66642705ee57fa03c5af5043d78260528448ec429b95e7c2547db1e3716d
Modified          : a77516c3a66ba61957b0e691cbe3058dc88d393e904f735a3216eb9f7a66cf5a
Bits changed      : 142/256 (55.47%)

--- HMAC-SHA256 ---
Original tag      : b95e29a4f15a7bb5768d4d74d301669005f9c37bdb2720780c93d2a74596dfb0
Modified tag      : 35edddb3e3907f7c44681b6f7527cf288d23ebba9e0da02da0a21b7b775b8357
Bits changed      : 116/256 (45.31%)

--- HMAC verification (receiver holds the key) ---
Original message + original tag : PASS
Modified message + original tag : FAIL

--- Attacker modifies the message and replaces the checksum ---
Attacker's SHA-256 : a77516c3a66ba61957b0e691cbe3058dc88d393e904f735a3216eb9f7a66cf5a
Receiver hash check : PASS (tampering NOT detected)
Attacker's HMAC tag : 7bbbb4fe9aaae9341a0fd34c5777550fa350d5aaff9882c1e7168459e36dfff5
Receiver HMAC check : FAIL (tampering detected)

--- Summary ---
One changed character changes about half the bits of both outputs.
HMAC verifies the original message and rejects the modified one.
A plain hash can be recomputed by anyone, so it cannot prove who
sent the message. HMAC needs the secret key, so only a key holder
can produce a valid tag: it gives integrity AND authentication.
```

What changed: one character flipped 142/256 bits of the hash and 116/256 bits of the tag, roughly half (the avalanche effect again). HMAC verification **passed** for the original message and **failed** for the modified one.

Why a hash is not enough: SHA-256 has no secret, so an attacker can change the message, recompute its hash and replace it. Our run shows the receiver accepting this forgery. **HMAC** mixes in a secret key, so without the key the attacker cannot produce a valid tag, and our run shows the forgery rejected. A valid tag proves **integrity** (the message is unchanged) and **authentication**. It does not give non-repudiation, since both sides share the key.

Task 5: Reflection

The most important lesson we learned is that a strong cipher does not make a system secure on its own. How it is used matters just as much.

Our avalanche experiment showed that AES itself is very strong. Flipping a single plaintext bit changed 49.9% of the ciphertext bits on average, almost exactly the ideal 50%. But Task 3 showed that this strength depends on correct use. AES-CBC and AES-CTR both recovered our message exactly, and each encryption used a fresh random IV or nonce, so the same message never produced the same ciphertext. Reusing a CTR nonce with the same key would reveal the XOR of two plaintexts without anyone breaking

AES. We also saw that encryption only hides content. Neither mode detects a modified ciphertext, and CTR even lets an attacker flip chosen plaintext bits.

Task 4 completed the picture. An attacker defeated a plain SHA-256 hash simply by recomputing it, but HMAC-SHA256 rejected the tampered message because the attacker did not have the secret key.

Secure communication therefore needs confidentiality and integrity together, plus careful handling of keys, IVs and nonces. This is why authenticated encryption such as AES-GCM is preferred in practice.